BACKEND DEVELOPER TRAINING · HOMEWORK

Event Tickets API

How the project was created, how the architecture connects, how a request travels from the client to the database, how security, configuration and Swagger are set up, how AI was used, how to demo it, and how to answer your mentor's questions.

Spring Boot 4.1 · Java 21 · Gradle (Groovy) · WAR · PostgreSQL · JPA / Hibernate · Flyway · JWT HS256 · Swagger / OpenAPI

1 Requirements
2 Creating the project
3 Database design
4 Architecture
5 Request flow: client → DB
6 Security and JWT
7 The six business rules
8 Error handling
9 Configuration
10 Swagger
11 Testing
12 Problems solved
13 Running the project
14 Mentor Q&A (short)
15 AI workflow and implementation
16 **Live demo with the demo page**
17 **Mentor questions explained**

Course: Backend Development with Spring Boot · KOSIGN Cambodia
Prepared by: _____ · Date: 13 Oct 2026
Note: the code was generated with the help of an AI assistant (Claude), then reviewed, run and tested by me (see section 15).

1 · Requirements

The project follows two slides of the course deck.

Slide 22: tech stack and conventions

- Spring Boot 4, Java 21, Gradle (Groovy DSL), **WAR** packaging, PostgreSQL, JPA, Flyway.
- Login with **JWT** (HS256, 1 hour expiry, secret from an environment variable); passwords stored with **BCrypt**.
- Roles **USER** and **ADMIN**: both can read; only ADMIN can create, update or delete.
- DB conventions: **uppercase abbreviated column names**, datetimes as `VARCHAR(14)` in `YYYYMMDDHH24MISS`, audit columns `REG_DTM / MOD_DTM / REG_ID / MOD_ID`.
- DTOs, a global JSON error handler, MockMvc tests for role checks.

Slide 23: business rules

#	Rule	Meaning	Result
1	No overbooking	Booking more tickets than seats left is refused	409
2	Seats stay correct	Booking reduces seats; cancelling gives them back	n/a
3	Max 4 tickets per booking	Quantity must be 1 to 4	400
4	Price from the server	Total = event price × quantity; never trust the client's price	n/a
5	No booking a cancelled event	Event status CANCELLED blocks booking	409
6	Only your own bookings	Customers view/cancel only their own bookings	403

> **What to say:** "Before coding, I turned every rule into an HTTP result: 400 for bad input, 403 for no permission, 409 when the request is valid but conflicts with the current state."

Endpoints

Method	Path	Who
POST	`/api/auth/login`	public
GET	`/api/events`, `/api/events/{id}`	USER, ADMIN
POST / PUT	`/api/events`, `/api/events/{id}`, `/api/events/{id}/cancel`	ADMIN
POST	`/api/bookings`	USER, ADMIN
GET	`/api/bookings`, `/api/bookings/{id}`	owner (ADMIN sees all)
POST	`/api/bookings/{id}/cancel`	owner or ADMIN

2 · Creating the project for the first time

2.1 Spring Initializr (start.spring.io)

Setting	Value	Why
Project	Gradle - Groovy	Build tool: downloads libraries, compiles, tests, packages
Spring Boot	4.1.x	Latest stable; avoid SNAPSHOT / M / RC
Group / Artifact	com.kosign / eventtickets	Becomes the base package `com.kosign.eventtickets`
Packaging	War	Required by the demo prompt; can still run with `java -jar`
Java	21	Long-term support release
Configuration	YAML	`application.yml` is easier to read

2.2 Dependencies and what each one gives

Dependency	Gives you
Spring Web (webmvc)	Embedded Tomcat, `@RestController`, JSON conversion (Jackson)
Spring Data JPA	Hibernate ORM and `JpaRepository` (CRUD without writing SQL)
Validation	`@NotBlank`, `@Min`, `@Max`, `@Valid`
Spring Security	Security filter chain, `SecurityContext`, BCrypt
OAuth2 Resource Server	JWT creation (encoder) and verification (decoder)
PostgreSQL Driver	JDBC connection to PostgreSQL
Flyway	Versioned SQL migrations that create and seed the tables
Lombok	`@Getter`, `@RequiredArgsConstructor`: less boilerplate
H2 (test only)	In-memory database so tests run without PostgreSQL
springdoc-openapi 3.x	Swagger UI generated from the controllers

2.3 Generated structure

```
eventtickets/
├── build.gradle          ← plugins + dependencies
├── settings.gradle       ← project name
├── gradlew, gradle/      ← Gradle wrapper: same Gradle version on every machine
├── compose.yaml          ← PostgreSQL in Docker
└── src/
    ├── main/java/com/kosign/eventtickets/
    │   ├── EventTicketsApplication.java   ← main(), @SpringBootApplication
    │   └── ServletInitializer.java        ← WAR only: boots inside external Tomcat
    ├── main/resources/
    │   ├── application.yml                ← configuration
    │   └── db/migration/V1__..., V2__...  ← Flyway SQL
    └── test/...                           ← MockMvc tests + application-test.yml
```

About `@SpringBootApplication`: it turns on **component scanning** (finds every `@RestController`, `@Service`, `@Repository` under the base package), **auto-configuration** (sees the PostgreSQL driver and creates a DataSource, sees Flyway and runs the migrations), and **configuration** (the class may define beans).

2.4 Order of work after generating

1. Start PostgreSQL with Docker (`docker compose up -d`).
2. Write `application.yml`: datasource, JPA, JWT.
3. Write Flyway migrations: tables (V1) and seed data (V2).
4. Create the packages (config, controller, service, repository, domain, dto, mapper, exception); build entities → repositories → mappers → services → controllers.
5. Security (login + JWT), then the global error handler.
6. Business rules in bookings, then tests, then Swagger.

3 · Database design

```
TB_USER  1 ──< TB_BOOKING >── 1  TB_EVENT
(who)          (who booked what)     (seats, price, status)
```

Table	Columns
`TB_USER`	`USR_ID` PK, `USR_NM` unique, `PWD` (BCrypt hash), `ROLE_CD` (USER/ADMIN)
`TB_EVENT`	`EVT_ID` PK, `EVT_NM`, `EVT_DTM` VARCHAR(14), `VENUE_NM`, `PRICE` NUMERIC(12,2), `TOT_SEAT`, `AVL_SEAT`, `STS_CD` (ACTIVE/CANCELLED)
`TB_BOOKING`	`BKG_ID` PK, `EVT_ID` FK, `USR_ID` FK, `QTY`, `UNIT_PRC`, `TOT_AMT`, `STS_CD` (BOOKED/CANCELLED)
every table	`REG_DTM`, `MOD_DTM` VARCHAR(14) · `REG_ID`, `MOD_ID` VARCHAR(50)

The database also protects the rules

```
CHECK (QTY BETWEEN 1 AND 4)        -- rule 3
CHECK (AVL_SEAT >= 0)                -- rule 1: seats can never go negative
CONSTRAINT CK_EVT_SEAT CHECK (AVL_SEAT <= TOT_SEAT)
CREATE INDEX IX_BKG_USR ON TB_BOOKING (USR_ID)   -- fast "my bookings"
```

Flyway migrations

- `V1__create_tables.sql` creates the three tables, constraints and indexes.
- `V2__seed_data.sql` adds users **admin/admin123**, **alice/user123**, **bob/user123** (two customers, to test rule 6) and three events: 100 seats, 5 seats (to test overbooking) and one already **CANCELLED** (to test rule 5).
- Flyway runs each file **once, in order**, and remembers what ran in `flyway_schema_history`. To change the schema later, add `V3__...`; never edit a migration that already ran.
- `ddl-auto: validate`: Hibernate does not create tables; it only checks the entities match the schema.

> **What to say:** "The rules are checked in Java and again by database constraints as a safety net. The schema is versioned with Flyway so every environment gets the same tables."

Why store `UNIT_PRC` in a booking? If the admin changes the event price later, old bookings keep the price the customer actually paid.

4 · Architecture

4.1 Layered architecture (slide 9)

Controller	`EventController` · `BookingController` · `AuthController`	HTTP in/out, `@Valid`, DTOs, status codes
▼ calls		
Service	`EventService` → `EventServiceImpl` `BookingService` → `BookingServiceImpl`	interface + impl: use cases, `@Transactional`, permission checks; uses a **Mapper** for DTOs
▼ uses		
Domain (Entity)	`Event.reserveSeats()` · `Booking.cancel()`	the business rules live here
▼ loaded / saved by		
Repository	`EventRepository` · `BookingRepository` · `UserRepository`	JpaRepository: Spring writes the SQL
▼ SQL		
Database	`PostgreSQL`	schema managed by Flyway

Each layer only calls the layer below it. Controllers never talk to repositories.

4.2 How the layers connect: dependency injection

No class does `new BookingService()`. At startup Spring creates one object (a **bean**) of every `@RestController`, `@Service` and repository and passes them in through the constructor:

```
@RestController
@RequiredArgsConstructor                        // Lombok writes the constructor
public class BookingController {
    private final BookingService bookingService;  // injected by Spring
}

public interface BookingService {               // service/  : WHAT it can do
    BookingResponse book(CurrentUser me, CreateBookingRequest request);
}

@Service
@RequiredArgsConstructor
public class BookingServiceImpl implements BookingService {   // service/impl/ : HOW
    private final BookingRepository bookingRepository;
    private final EventRepository eventRepository;
    private final UserRepository userRepository;
    private final BookingMapper bookingMapper;
}
```

The controller depends on the **interface** `BookingService`; Spring injects the only implementation, `BookingServiceImpl`. The controller never knows which class does the work, so the implementation can change (or be mocked in a test) without touching the controller.

4.3 DTO vs Entity

```
Client JSON  ⇄  DTO (record)          ⇄  Mapper         ⇄  Entity   ⇄  Table
              dto/request/                 mapper/            domain/entity/
              CreateBookingRequest         BookingMapper      Booking      TB_BOOKING
              dto/response/
              BookingResponse
```

- Entities never leave the service; `BookingMapper.toResponse(booking)` converts them. DTOs are plain records with no logic.
- Hides internal data (password hash, audit columns) and keeps the API stable when tables change.
- `CreateBookingRequest` has **no price field**, so the client physically cannot send a price (rule 4).

4.4 Package structure: layer + feature

```
com.kosign.eventtickets
├── EventTicketsApplication, ServletInitializer
├── config/          SecurityConfig, OpenApiConfig, JwtProperties
├── controller/      AuthController, EventController, BookingController
├── service/         AuthService, EventService, BookingService        ← interfaces
│   └── impl/        AuthServiceImpl, EventServiceImpl, BookingServiceImpl
├── repository/      UserRepository, EventRepository, BookingRepository
├── domain/entity/   BaseEntity, AppUser, Event, Booking
│   └── enums/       Role, EventStatus, BookingStatus
├── dto/
│   ├── request/     LoginRequest, EventRequest, CreateBookingRequest
│   └── response/    TokenResponse, EventResponse, BookingResponse
├── mapper/          EventMapper, BookingMapper                    ← entity ⇄ DTO
├── exception/       GlobalExceptionHandler, NotFoundException, ConflictException
├── security/        CurrentUser                                   ← caller from the JWT
└── util/            DateTimes                                     ← VARCHAR(14) date format
```

Package	Responsibility
`controller`	HTTP only: URL, validation, status code. No business logic.
`service` / `impl`	Use cases and transactions. Interface = contract, impl = code.
`repository`	Database access through Spring Data JPA.
`domain.entity`	Tables as Java classes, plus the business rules.
`dto.request` / `response`	What the API receives and returns.
`mapper`	Converts entities to DTOs (and requests to entities).
`exception`	Custom exceptions and the global JSON error handler.
`config` / `security` / `util`	Setup beans, JWT caller info, helpers.

4.5 Rich domain model: rules inside the entity

```java
// domain/entity/Event.java
public void reserveSeats(int quantity) {
    if (isCancelled())                                  // rule 5
        throw new ConflictException("Event " + id + " is cancelled");
    if (quantity > availableSeats)                      // rule 1
        throw new ConflictException("Only " + availableSeats + " seat(s) left");
    availableSeats -= quantity;                          // rule 2
}
```

The entity has no public setters, so nobody can put it in an invalid state from outside.

4.6 BaseEntity: audit columns filled automatically

```
@MappedSuperclass
public abstract class BaseEntity {
    @PrePersist  void onCreate() { regDtm = modDtm = now(); regId = modId = currentUsername(); }
    @PreUpdate   void onUpdate() { modDtm = now(); modId = currentUsername(); }
}   // now() → "20260928174327", currentUsername() → from SecurityContext, or "system"
```

4.7 Is this "Clean Architecture"?

> **Honest answer:** "It is a **layered architecture organized by feature**, the pattern from the course. It shares Clean Architecture's key ideas: business rules sit in the domain, controllers don't touch the database, and layers connect through dependency injection. Strict Clean Architecture goes further: the domain would be pure Java with no JPA/Spring annotations, and services would depend on interfaces (*ports*) implemented by the database layer (*adapters*). For a project this size that adds code without much benefit."

5 · Request flow: client → API → database

Example: **alice books 2 tickets** for event 1.

```
POST /api/bookings
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{ "eventId": 1, "quantity": 2 }
```

1. **Tomcat receives the HTTP request**
   Embedded server on port 8080 passes it into the Spring application.

2. **Security filter chain (before any of our code)**
   a. `BearerTokenAuthenticationFilter` reads the `Authorization` header
   b. `JwtDecoder` checks the HS256 signature and the expiry
   c. `JwtAuthenticationConverter` turns claim `roles: ["USER"]` into `ROLE_USER`
   d. the user is stored in the `SecurityContext`
   e. `AuthorizationFilter` : `/api/bookings/**` needs USER or ADMIN ✔
   ✘ no / bad / expired token → 401 · wrong role → 403

3. **DispatcherServlet → BookingController.book()**
   Jackson converts JSON → `CreateBookingRequest` ; `@Valid` checks `quantity` is 1..4.
   ✘ invalid input → 400 with field errors

4. **Controller**
   `CurrentUser.from(jwt)` → id 2, "alice", not admin; calls `bookingService.book(me, request)` .

5. **BookingServiceImpl: `@Transactional` → BEGIN**
   `eventRepository.findByIdForUpdate(1)`
   `SELECT ... FROM TB_EVENT WHERE EVT_ID = 1 FOR UPDATE` 🔒 `row locked`
   `Booking.create(event, user, 2)` → `event.reserveSeats(2)` ; total = price × 2
   ✘ cancelled event / not enough seats → 409
   `bookingRepository.save(booking)` → `@PrePersist` fills REG_DTM, REG_ID = "alice"
   `INSERT INTO TB_BOOKING (EVT_ID, USR_ID, QTY, UNIT_PRC, TOT_AMT, STS_CD, ...) VALUES (...)`
   COMMIT: Hibernate notices `availableSeats` changed (dirty checking)
   `UPDATE TB_EVENT SET AVL_SEAT = 98, MOD_DTM = ..., MOD_ID = 'alice' WHERE EVT_ID = 1`

6. **Response**
   `BookingMapper.toResponse()` builds the `BookingResponse` DTO → Jackson → JSON with
   201 Created `{ "id": 1, "quantity": 2, "unitPrice": 25.00, "totalAmount": 50.00, "status": "BOOKED" }`

Key points to explain

- **Security runs first.** A request without a valid token never reaches the controller.
- `@Transactional` : seats are reduced *and* the booking is saved, or neither is (rollback on any exception).
- `FOR UPDATE` : a second person booking the same event waits until the first transaction commits.
- **Dirty checking**: we never call `save(event)` ; Hibernate writes the UPDATE because the managed entity changed.
- **Error path**: any `ConflictException` rolls back and `GlobalExceptionHandler` returns a 409 JSON body.

6 · Security and JWT (slides 18–20)

6.1 Login

1. `POST /api/auth/login` with username + password.
2. `AuthService` finds the user and checks the password with `passwordEncoder.matches()` (BCrypt).
3. Wrong username and wrong password give the **same** 401 message, so attackers can't discover valid usernames.
4. A JWT is created and signed with HS256:

```
{ "sub": "alice", "uid": 2, "roles": ["USER"], "iat": 1790000000, "exp": 1790003600 }   // valid 1 hour
```

The client sends it on every request: `Authorization: Bearer <token>`. The server keeps no session: **stateless**.

6.2 SecurityFilterChain

```
http
  .csrf(csrf -> csrf.disable())                      // token in header, no cookies → no CSRF risk
  .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers("/api/auth/**", "/error").permitAll()
      .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
      .requestMatchers(GET, "/api/events/**").hasAnyRole("USER", "ADMIN")
      .requestMatchers("/api/events/**").hasRole("ADMIN")
      .requestMatchers("/api/bookings/**").hasAnyRole("USER", "ADMIN")
      .anyRequest().authenticated())
  .oauth2ResourceServer(o -> o
      .bearerTokenResolver(bearerTokenResolver())     // ignore tokens on /api/auth/**
      .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
```

6.3 Security beans

Bean	Job
`PasswordEncoder` (BCrypt)	Hash and check passwords
`SecretKey`	HS256 key built from `JWT_SECRET` (must be ≥ 32 bytes, else app won't start)
`JwtEncoder`	**Creates** tokens at login
`JwtDecoder`	**Verifies** tokens on every request
`JwtAuthenticationConverter`	Maps the `roles` claim to `ROLE_USER` / `ROLE_ADMIN` so `hasRole()` works
`BearerTokenResolver`	Skips token reading on public login, so an old token can't block login

Authentication: who are you?

Fails → **401 Unauthorized**

no token, bad signature, expired, wrong password

Authorization: what may you do?

Fails → **403 Forbidden**

USER creating an event, bob opening alice's booking

7 · The six business rules: how and why

Rule	How it is implemented	Why this way
3. 1–4 tickets	`@Min(1) @Max(4)` on `CreateBookingRequest.quantity` → `400`; DB `CHECK` too	Format rules are validated at the edge (slide 16)
4. Server price	Request has no price field; `totalAmount = event.price × quantity` in `Booking.create()`	Never trust the client. A test sends a fake price and checks it is ignored
5. Cancelled event	`Event.reserveSeats()` checks status → `409`	409 = request valid, but conflicts with current state
1. No overbooking	`reserveSeats()` checks seats left → `409`, plus a **row lock**	Stops the race condition below
2. Seats correct	Book: `availableSeats -= qty`. Cancel: `releaseSeats(qty)`. Cancel twice → `409`	Otherwise a double cancel would return seats twice
6. Own bookings	`BookingServiceImpl.findAccessible()`: not owner and not admin → `AccessDeniedException` → `403`	Admins can see all for support work

⭐ The race condition (rule 1 under load)

> **Problem:** 1 seat left. Alice and Bob click "book" at the same millisecond. Both read "1 seat left", both pass the check, and the event is **oversold**.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)                    // EventRepository
@Query("select e from Event e where e.id = :id")
Optional<Event> findByIdForUpdate(@Param("id") Long id);   // → SELECT ... FOR UPDATE
```

Fix: the event row is locked while seats are reserved. The second request **waits** until the first commits, then sees 0 seats and gets 409. `ConcurrentBookingTest` proves it: **20 parallel bookings for 5 seats → exactly 5 succeed, 15 get 409**.

> **Alternative:** optimistic locking (`@Version` column + retry). Pessimistic was chosen because popular events have many conflicts, and waiting is simpler than retrying.

N + 1 query problem (slide 13)

The booking list shows each event's name. With lazy loading, 20 bookings could run 1 + 20 queries. `@EntityGraph(attributePaths = {"event", "user"})` on the repository loads everything in **one query with joins**. `default_batch_fetch_size: 50` is a second safety net.

8 · Error handling (slide 17)

`GlobalExceptionHandler` (`@RestControllerAdvice`) turns every error into a standard **ProblemDetail** (RFC 9457) JSON body.

Exception	Status	Example
`MethodArgumentNotValidException`	400	quantity = 5, with an `errors` map per field
`PropertyReferenceException`	400	`?sort=password` (unknown sort field)
`AuthenticationException`	401	no token / wrong password
`AccessDeniedException`	403	bob opens alice's booking
`NotFoundException`	404	event 999
`ConflictException`	409	no seats left, event cancelled
`Exception`	500	logged on the server; client never sees a stack trace

```
{
  "title": "Conflict", "status": 409,
  "detail": "Only 1 seat(s) left for event 2",
  "instance": "/api/bookings"
}
```

Detail worth explaining: 401/403 happen inside the **security filter**, before any controller, so they never reach the advice by themselves. The `authenticationEntryPoint` and `accessDeniedHandler` forward them to `HandlerExceptionResolver`, so every error has the same JSON shape.

9 · Configuration

9.1 build.gradle

```
plugins {
    id 'java'
    id 'war'                                                // package as WAR
    id 'org.springframework.boot' version '4.1.1'           // bootRun, bootWar
    id 'io.spring.dependency-management' version '1.1.7'  // compatible versions from Boot BOM
}
dependencyManagement { applyMavenExclusions = false }     // fix: WAR build lost spring-web
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-security'
    implementation 'org.springframework.boot:spring-boot-starter-security-oauth2-resource-server'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:3.1.1'
    runtimeOnly    'org.postgresql:postgresql'
    providedRuntime 'org.springframework.boot:spring-boot-starter-tomcat'  // external Tomcat
provides it
    testRuntimeOnly 'com.h2database:h2'
}
```

9.2 application.yml

```
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/eventtickets}   # env var, or default
    username: ${DB_USERNAME:eventtickets}
    password: ${DB_PASSWORD:eventtickets}
  jpa:
    open-in-view: false            # DB access only inside services
    hibernate.ddl-auto: validate   # Flyway creates tables, Hibernate only checks
    properties.hibernate.default_batch_fetch_size: 50
  data.web.pageable:
    max-page-size: 100             # nobody can ask ?size=1000000
    serialization-mode: via-dto    # stable page JSON: content + page
app:
  jwt:
    secret: ${JWT_SECRET}          # no default: must come from the environment
    expiry: 1h
springdoc:
  swagger-ui.persist-authorization: true
```

- `${NAME:default}` reads an environment variable; secrets stay out of the code.
- `app.jwt.*` is bound to the `JwtProperties` record (`@ConfigurationProperties`), which fails at **startup** if the secret is too short.
- `src/test/resources/application-test.yml` overrides the datasource with H2 and sets a test secret (`@ActiveProfiles("test")`).

10 · Swagger (springdoc-openapi)

How it works

```
App starts → springdoc scans all @RestController classes
          → builds an OpenAPI description (URLs, parameters, DTO fields, validation rules)
          → serves it as JSON at        /v3/api-docs
          → Swagger UI draws it at      /swagger-ui.html
```

The documentation is **generated from the code**, so it can never be out of date.

The five configuration points

#	Where	What
1	`build.gradle`	`springdoc-openapi-starter-webmvc-ui:3.1.1` (3.x is the line for Spring Boot 4)
2	`OpenApiConfig.java`	Title, and the JWT **Authorize** button
3	`SecurityConfig`	`permitAll()` for `/swagger-ui/**` and `/v3/api-docs/**`, otherwise Swagger itself returns 401
4	Controllers	`@SecurityRequirements` on `AuthController` (login needs no token); `@ParameterObject` on `Pageable` (shows page / size / sort fields)
5	`application.yml`	`persist-authorization: true` keeps the token after refresh

```java
@Configuration
@OpenAPIDefinition(
    info = @Info(title = "Event Tickets API", version = "v1"),
    security = @SecurityRequirement(name = "bearerAuth"))        // all endpoints need a token
@SecurityScheme(name = "bearerAuth", type = SecuritySchemeType.HTTP,
                scheme = "bearer", bearerFormat = "JWT")          // adds the Authorize button
public class OpenApiConfig { }
```

Using Swagger

1. Open `http://localhost:8080/swagger-ui.html`.
2. **POST /api/auth/login** → Try it out → `{"username":"alice","password":"user123"}` → copy `accessToken`.
3. Click **Authorize** 🔒 and paste **only** the token (no "Bearer ", Swagger adds it).
4. Every endpoint now works with **Try it out**. To switch user: Authorize → Logout → log in again.

> Opening `http://localhost:8080/` in a browser returns 401. That is correct: this is a REST API, not a website, and every URL except login needs a token.

11 · Testing

- **MockMvc** sends real HTTP requests through the full stack (security → controller → service → DB) without starting a server.
- Tests run on **H2 in PostgreSQL mode**, so no Docker is needed. The same Flyway migrations run in the tests.
- `@Transactional` on the test class rolls back after each test, so tests don't affect each other.
- **20 tests, all passing**:

Area	Tests
Auth	wrong password → 401 · no token → 401 · garbage token → 401 · login ignores a stale token
Roles	USER lists events ✔ · USER creates event → 403 · ADMIN creates event → 201
Validation	invalid event → 400 with field errors · unknown sort → 400 · unknown event → 404
Rule 1	4 of 5 seats OK, then 2 more → 409 · **20 parallel bookings for 5 seats → exactly 5 succeed**
Rule 2	book 3 → 97 seats · cancel → 100 · cancel again → 409, still 100
Rule 3	quantity 0 and 5 → 400 · quantity 4 → 201
Rule 4	client sends `totalAmount: 0.01` → server charges 75.00
Rule 5	book cancelled event → 409 · admin cancels event, then booking → 409
Rule 6	bob views / cancels alice's booking → 403 · bob's list is empty · admin sees all
Extra	admin can't shrink seats below booked seats → 409

```
./gradlew test          # BUILD SUCCESSFUL: 20 tests, 0 failures
```

12 · Problems found and solved

1. start.spring.io failed for Gradle projects

The Initializr server returned 500 for every Gradle project that day (Maven worked). **Solution:** wrote `build.gradle` by hand, using the generated Maven pom to learn the exact Spring Boot 4 artifact names; created the Gradle wrapper locally.

2. App would not start: `ClassNotFoundException: HttpMessageConverters`

Debugging: printed the test classpath and found `spring-web` missing. The Tomcat runtime starter has a wildcard Maven exclusion, and with WAR packaging the dependency-management plugin applied it too broadly. **Solution:** `dependencyManagement { applyMavenExclusions = false }`.

3. Tests affected each other

The concurrency test must really commit (no rollback), and its rows leaked into the other tests. **Solution:** gave it its own in-memory H2 database.

4. Unknown sort field returned 500

`?sort=password` threw `PropertyReferenceException`. **Solution:** a handler returning 400.

5. Swagger login returned 401

Swagger sent an old / broken token on the login request. Spring Security validates any token it receives, even on `permitAll()` URLs. **Solution:** a `BearerTokenResolver` that ignores tokens on `/api/auth/**`, `@SecurityRequirements` on the login controller, and a regression test.

6. Port 5432 already used

Another project's PostgreSQL container used port 5432. **Solution:** ran this project's database on port 5433 and passed `DB_URL` as an environment variable, without changing any project files.

13 · Running the project

First time

```
# 1. Start PostgreSQL (port 5433 because 5432 is used by another project)
docker run -d --name eventtickets-pg -e POSTGRES_DB=eventtickets \
  -e POSTGRES_USER=eventtickets -e POSTGRES_PASSWORD=eventtickets -p 5433:5432 postgres:17

# 2. Run the app
cd eventtickets
DB_URL=jdbc:postgresql://localhost:5433/eventtickets \
JWT_SECRET=local-dev-secret-please-change-0123456789abcdef \
./gradlew bootRun
```

Next times

```
docker start eventtickets-pg
DB_URL=jdbc:postgresql://localhost:5433/eventtickets JWT_SECRET=... ./gradlew bootRun
```

Useful commands

Command	Does
`./gradlew test`	Run all 20 tests (H2, no Docker needed)
`./gradlew bootWar`	Build `build/libs/eventtickets-0.0.1-SNAPSHOT.war`
`java -jar build/libs/*.war`	Run the WAR with embedded Tomcat
`docker stop eventtickets-pg`	Stop the database

Quick check with curl

```
TOKEN=$(curl -s localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"user123"}' | sed -E 's/.*"accessToken":"([^"]+)".*/\1/')

curl -s localhost:8080/api/events -H "Authorization: Bearer $TOKEN"

curl -s localhost:8080/api/bookings -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"eventId":1,"quantity":2}'
```

Verified live on PostgreSQL

Check	Result
Book 3 × 25.00 while sending a fake price	`201` total 75.00
Quantity 5	`400` "must be between 1 and 4"
4 of 5 seats, then 2 more	`201` then `409` "Only 1 seat(s) left"
Book the cancelled event	`409` "Event 3 is cancelled"
Bob opens alice's booking	`403`
Alice cancels 3 tickets	seats 97 → 100
Audit columns in DB	`REG_DTM = 20260928174327`, `REG_ID = alice`

14 · Questions a mentor may ask

Question	Answer
Why JWT, not sessions?	A REST API for web and mobile clients: stateless, no session storage on the server, easy to scale.
Why HS256?	One service both creates and checks tokens, so one shared secret is enough. RS256 (public/private key) is better when other services must verify tokens.
Why is CSRF disabled?	CSRF attacks abuse cookies. This API uses a token in the Authorization header, not cookies.
Why 403, not 404, for someone else's booking?	The homework asks for 403. Some teams return 404 so they don't reveal the ID exists; that is a valid trade-off.
Why a pessimistic lock?	Popular events cause many conflicts; waiting is simpler than retrying. Optimistic locking (`@Version`) + retry would also work.
What is `@Transactional` for?	All-or-nothing: seat update and booking insert both commit, or both roll back.
Why service interface + impl?	The controller depends on a contract, not a class: easy to mock in tests and to swap the implementation. It is also the common team convention.
Why a separate mapper?	DTOs stay plain data, and entity ⇄ DTO conversion lives in one place per feature instead of being spread over services and DTOs.
Why put logic in entities?	Rules live in one place and can't be skipped by accident (rich domain model).
Why DTOs?	Hide internal fields, keep the API stable, and control input (no price field → rule 4).
Why Flyway, not `ddl-auto: update` ?	Versioned, repeatable, reviewable schema changes. `update` is unsafe in production.
Why `open-in-view: false` ?	Database access stays inside the service/transaction; prevents hidden lazy queries while writing JSON.
How do you avoid N + 1?	`@EntityGraph` on list queries, plus `default_batch_fetch_size` .
Why store `UNIT_PRC` ?	Old bookings keep the price actually paid if the event price changes.
Why WAR?	The requirement (slide 22). `ServletInitializer` lets it run in an external Tomcat; it still runs with `java -jar` .
What would you add next?	Refresh tokens, cancelling all bookings when an event is cancelled, Testcontainers for PostgreSQL tests, request logging.

Tip: put breakpoints in `BookingController.book()` , `BookingServiceImpl.book()` and `Event.reserveSeats()` , then book from Swagger and step through with F8. Seeing section 5 happen live is the best way to learn it.

15 · AI workflow and implementation

15.1 Tools used

Tool	Used for
Claude Code (AI coding assistant in the terminal)	Reading the slides, generating the project, writing code and tests, running commands, debugging, and writing explanations
Spring Initializr	Project settings and Spring Boot 4 library names
Gradle, JUnit, MockMvc, H2	Building and automated testing
Docker + PostgreSQL 17	Running the real database locally
Swagger UI, curl	Manual testing of every endpoint

15.2 How the work went, step by step

● my request or check ● AI work

1 **Give the AI the requirements**

Prompt: "Read this slide" (the course .pptx), then "continue"

AI extracted the text of all 24 slides and found the demo prompt (slide 22) and the homework rules (slide 23).

Lesson: "continue" was too vague. The AI assumed I wanted the full homework built. Clear prompts matter.

2 **AI builds the first version**

Created the Gradle project, Flyway migrations, entities, security with JWT, the six rules, the error handler and 19 tests. Built the WAR.

Problems it hit and fixed: Initializr returned 500 for Gradle projects (wrote build.gradle by hand); app would not start because spring-web was missing (found by printing the classpath); tests affected each other (separate test database).

3 **I asked for guidance instead of only answers**

Prompt: "Just guide how to create a backend project and do the homework step by step"

AI wrote a 10-step guide: full setup code, but only hints for the business rules, so I could learn them.

4 **Run it for real**

Prompt: "Can you please run this project?"

AI started PostgreSQL in Docker (port 5433, because another project already used 5432), ran the app, and tested every rule with curl. It also checked the audit columns in the database.

5 **I asked why the browser shows 401, and asked for Swagger**

AI explained it's an API that needs a token, then added springdoc Swagger with a JWT Authorize button.

6 **I found a bug while testing in Swagger**

I sent a screenshot: login returned 401

AI reproduced it with curl: Swagger sent an old or broken token on the login request, and Spring Security rejects bad tokens even on public URLs. It fixed the server and Swagger config and added a regression test.

7 **I reviewed the structure and asked for it to be cleaner**
Prompt: "Structure is not clean yet, it should have a clean package structure"
AI offered three options with previews; I chose layer + feature. It moved every class, added service interfaces + impl and mapper classes.
AI mistake: its file-moving script emptied the API test file. It restored it and re-ran all 20 tests (all passing).

8 **I asked the AI to explain everything I must present**
How the project was created, the architecture, the request flow, configuration, Swagger and database migrations. This PDF is the result.

9 **I checked the homework requirements again**
Business rules ✔, technical implementation ✔, and this AI workflow section and the demo script were added.

15.3 Who did what

The AI

- Read the slides and turned them into a plan
- Generated the project, SQL, Java code and tests
- Ran builds, tests and the app; tested with curl
- Debugged errors from logs and the classpath
- Explained the code and wrote this document

Me

- Gave the requirements and chose what to build
- Tested it myself in Swagger and found the login bug
- Reviewed the structure and asked for a cleaner one
- Chose the final package structure
- Studied every part so I can explain and change it

15.4 How the AI's work was checked

Check	What it proved
20 automated tests	Every business rule, every role check, and 20 parallel bookings never overselling. Re-run after every change.
Live run on PostgreSQL	Flyway migrations work on the real database; all rules return the right status codes.
Database check	Audit columns are filled correctly (`REG_DTM` , `REG_ID = alice`).
Manual Swagger testing	Found a real bug (login 401) that the tests had missed. A test was added for it.
Reading the code	Found the structure was not what the team expects, which led to the restructure.

15.5 Mistakes the AI made (and how they were caught)

Mistake	Caught by / fixed how
Built the whole homework from a vague "continue"	I questioned it; after that it guided me step by step instead
Swagger login could be blocked by an old token	My manual testing; fixed and covered by a new test
First package structure didn't match team style	My review; restructured to layer + feature

Script emptied a test file during the restructure; missed some imports	The compiler and test run; file restored, all tests pass

15.6 What I learned about working with AI

- **Clear prompts get better results.** "continue" led to work I didn't ask for; specific requests worked well.
- **AI makes mistakes, so verification matters.** Tests, a real run and manual testing each caught something different.
- **Tests make changes safe.** The big restructure was safe because 20 tests confirmed nothing broke.
- **Understanding the code is my job.** I asked for explanations of every part (setup, architecture, request flow, migrations) so I can defend and change it.
- **AI is fastest at boilerplate and debugging;** design choices (structure, what to build) still needed my decisions.

> **What to say:** "I used Claude Code as a pair programmer. It generated the code and I verified it with tests, a live run and manual Swagger testing. I found a bug and asked for a cleaner structure. Then I studied every part so I can explain it."

16 · Live demo with the demo page (about 7 minutes)

The app includes its own demo page. Each business rule has a tab with steps you click one by one; every step shows the real HTTP request, the response, and ✓ or ✗ against the expected status.

Start it (10 minutes before)

```
# 1. Open Docker Desktop, then start the database
docker start eventtickets-pg

# 2. Start the app (port 8090 because other projects use 8080/8081)
cd eventtickets
SERVER_PORT=8090 DB_URL=jdbc:postgresql://localhost:5433/eventtickets \
JWT_SECRET=local-dev-secret-please-change-0123456789abcdef ./gradlew bootRun

# 3. Open http://localhost:8090/
```

No database reset is needed: each rule creates its own new event first, so the demo works on any data and can be repeated.

What to click and what to say

Tab	Click (in order)	Say
01 No overbooking	Admin creates a 5-seat event → Alice books 4 → Bob books 2	"Only 1 seat is left, so Bob gets **409 Conflict**. The seat counter shows 1 / 5."
02 Seats stay correct	Create 10-seat event → Alice books 3 → cancel → cancel again	"Seats go 10 → 7 → 10. Cancelling twice is **409**, so seats can't come back twice."
03 Max 4 tickets	Create event → 5 tickets → 0 tickets → 4 tickets	"5 and 0 are **400 Bad Request**; 4 is accepted."
04 Price from server	Create event at 25.00 → Alice sends a fake price 0.01 for 3 tickets	"The server ignores my price and charges **75.00**."
05 Cancelled event	Create event → admin cancels it → Alice books	"Booking a cancelled event is **409**."
06 Own bookings	Create event → Alice books → Bob views → Bob cancels → Alice views	"Bob gets **403 Forbidden** on Alice's booking; Alice can see it."
+ Security	No login → Bob creates event → Admin creates event	"No token is **401**, a customer is **403**, admin is **201**."

Tips: point at the big result banner after each click. "Start this rule again" repeats a tab. The "Advanced" section at the bottom has the request history and a custom request box if your mentor asks for something else. If the page looks old, hard refresh with **Cmd + Shift + R**. If nothing works, run `./gradlew test` and use this PDF.

Finish with proof (1 minute)

```
./gradlew test   # 21 tests pass, incl. 20 parallel bookings on 5 seats → exactly 5 succeed
```

If something goes wrong

Problem	Fix
"Port 8090 was already in use"	Use another free port, e.g. `SERVER_PORT=8095`, and open that port
"Connection to localhost:5433 refused"	Docker Desktop is closed or the database is stopped: open Docker, `docker start eventtickets-pg`
Steps fail with 401 after a long time	Tokens expire after 1 hour: click a user chip at the top to log in again

17 · Mentor questions explained

Each card has a **simple answer**, then **in our project** (so you can point at real code), then a **remember** line you can say in one breath.

Part	Topics
A Requirements	BRD, PRD, SRS · where ours came from
B Database	Main tables, roles, PK vs unique key vs index, did you create indexes, offset vs keyset, CTE, transactions
C API	Process flow, path variable vs request param vs body, HTTP methods, status codes
D Querying data	How many ways to query, N + 1, LAZY vs EAGER
E Two requests at the same time	Race condition and 5 ways to handle it
F Security config	Authentication vs authorization, JWT, filter chain, BCrypt, CSRF
G Code structure and patterns	Packages, design patterns, rapid-fire answers

A Requirements documents

What the business wants → what the product does → how we build it

BRD WHY · business → **PRD** WHAT · features → **SRS / Tech spec** HOW · system → **Design** ERD, API spec → **Code** + tests → **Demo / UAT** check vs BRD

	BRD Business Requirements Document	**PRD** Product Requirements Document	**SRS / Technical spec** Software Requirements Spec
Answers	WHY are we building this? What business problem?	WHAT will the product do? Features, users, rules	HOW will the system do it? APIs, DB, tech
Written by	Business analyst, client, PO	Product owner / product manager	Tech lead / developers
Contains	Goals, scope, stakeholders, business rules, success measures	User roles, user stories, features, acceptance criteria, priorities	Architecture, ERD, API endpoints, security, performance
Our project	"Sell event tickets online without overselling, and charge the right price" + the 6 business rules (slide 23)	Roles USER / ADMIN, endpoints for events and bookings, each rule with its expected result (PDF section 1)	Slide 22: Spring Boot 4, PostgreSQL, JWT, DB conventions + our ERD and API list

Q Mini BRD of our project

- **Problem:** customers buy event tickets; an event must never sell more seats than it has.
- **Goal:** correct seat counts and correct money, always.
- **Stakeholders:** customers, event admins.
- **Business rules:** the 6 rules (overbooking, seats, max 4, server price, cancelled event, own bookings).
- **Success:** every rule returns the agreed result (409 / 400 / 403).

Q Mini PRD of our project

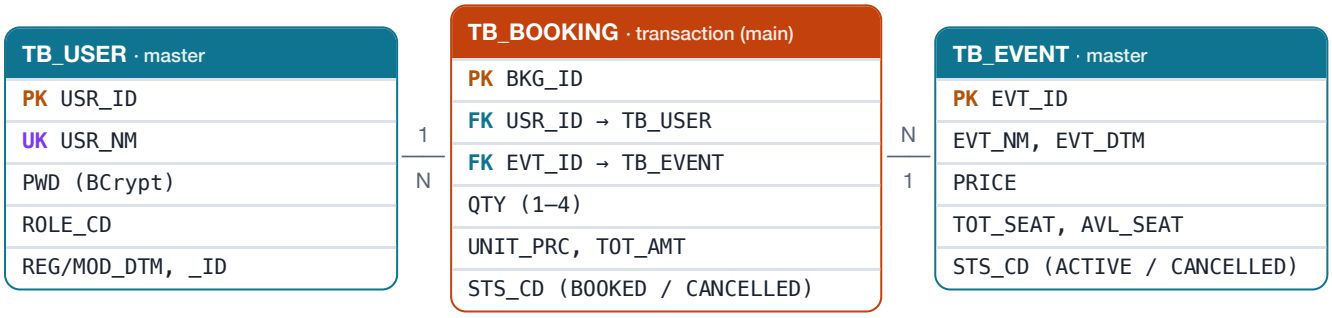
- **Users:** USER (customer) books tickets; ADMIN manages events.
- **Stories:** "As a customer I can book 1–4 tickets", "As an admin I can cancel an event".
- **Acceptance:** booking more than seats left → 409; quantity 5 → 400; another customer's booking → 403.
- **Out of scope:** payment, refunds, email.

If asked "Did you write a BRD/PRD?" "We didn't get formal documents. The slides were our requirements: slide 23 is the business rules (BRD level) and slide 22 is the technical spec. I turned them into a requirements table with an expected result for each rule (section 1), which is how a PRD's acceptance criteria work."

B Database

Tables, keys, indexes, pagination, CTE, transactions

Q What are the main tables?



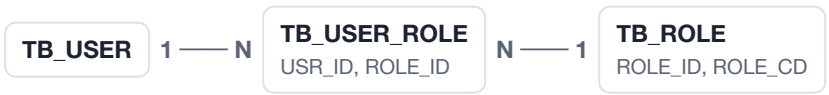
Master tables hold things that exist on their own and change rarely (users, events). A **transaction table** records what happens between them (a booking). **TB_BOOKING is the main business table**: every business rule touches it.

Remember: "Two master tables, one transaction table. One user has many bookings, one event has many bookings."

Q How do you store roles? Why no role table?

Each user has **one role** in `TB_USER.ROLE_CD` (`'USER'` or `'ADMIN'`), mapped to the Java enum `Role`. At login it goes into the JWT as `"roles": ["USER"]`, and Spring checks it with `hasRole("ADMIN")` (Spring adds the `ROLE_` prefix).

If a user needed many roles or permissions, normalize it:



Remember: "One role per user, so a column is enough. Many roles → a many-to-many table."

PK vs unique key vs index (and FK)

	Primary key (PK)	Unique key (UK)	Index	Foreign key (FK)
Purpose	Identifies each row	Business value must not repeat	Makes searching faster	Links to another table's PK
Duplicates?	Never	Never	Allowed (unless unique index)	Allowed (many bookings → 1 event)
NULL?	Never	Allowed (PostgreSQL allows several NULLs)	Allowed	Allowed unless NOT NULL
How many?	Exactly 1 per table	Many	Many	Many
Auto index?	yes	yes	it *is* one	no in PostgreSQL → add it yourself
Ours	`USR_ID`, `EVT_ID`, `BKG_ID`	`USR_NM` (username)	`IX_BKG_USR`, `IX_BKG_EVT`	`TB_BOOKING.USR_ID`, `.EVT_ID`

> **Remember:** "PK = who the row is. Unique key = a rule that a value can't repeat. Index = speed. Both PK and UK create an index automatically; an FK does not in PostgreSQL."

Q Did you create indexes? Which and why?

Yes. This is the real list from our database (`pg_indexes`):

```
    indexname       |  created by           | on column
--------------------+-----------------------+----------------------
 tb_user_pkey       | PostgreSQL (PK)       | TB_USER(USR_ID)
 tb_event_pkey      | PostgreSQL (PK)       | TB_EVENT(EVT_ID)
 tb_booking_pkey    | PostgreSQL (PK)       | TB_BOOKING(BKG_ID)
 tb_user_usr_nm_key | PostgreSQL (UNIQUE)   | TB_USER(USR_NM)       ← fast login lookup
 ix_bkg_usr         | us, in V1 migration   | TB_BOOKING(USR_ID)    ← "my bookings"
 ix_bkg_evt         | us, in V1 migration   | TB_BOOKING(EVT_ID)    ← bookings of an event
```

Why those two? They are foreign keys we search by (`WHERE USR_ID = ?`), and PostgreSQL does not index FKs automatically. Without an index the database reads every booking row (full scan); with it, it jumps straight to the matching rows.

Cost of an index: more disk space and slightly slower INSERT/UPDATE, because the index must be updated too. So index columns you search, join or sort by, not every column. Check with `EXPLAIN SELECT ...` (look for "Index Scan" vs "Seq Scan").

> **Remember:** "Index the columns in WHERE, JOIN and ORDER BY. PK and unique get one automatically; FK columns I add myself."

Q Pagination: offset vs keyset (cursor)

Offset (what we use: `?page=2&size=20`)

```
SELECT * FROM TB_EVENT
ORDER BY EVT_ID DESC
LIMIT 20 OFFSET 40;     -- page 3
```

Keyset ("give me 20 after the last one I saw")

```
SELECT * FROM TB_EVENT
WHERE EVT_ID < 30       -- last id on previous page
ORDER BY EVT_ID DESC
LIMIT 20;
```

	Offset	Keyset
Speed on page 10,000	Slow: database still reads and skips all earlier rows	Fast: index jumps straight to `EVT_ID < 30`
Rows added while paging	Can show a row twice or skip one	Stable
Jump to page 7 / total pages	Yes	No, only "next"
Best for	Admin tables with page numbers	Infinite scroll, feeds, big tables
In Spring	`Pageable` → `Page<T>` (ours)	`ScrollPosition.keyset()` → `Window<T>`

In our project: offset paging with `Pageable` (`max-page-size: 100`). Fine for small data; for a large booking history I'd switch to keyset. (Tested: the keyset query above returns events 29, 28, 27.)

> **Remember:** "Offset skips rows, keyset continues from the last row. Keyset is faster for big data but can't jump to a page."

Q What is a CTE?

A **CTE (Common Table Expression)** is a named temporary result written with `WITH name AS (...)` that you can use like a table in the rest of the query. It makes long SQL readable: step 1, step 2, final select. A *recursive* CTE can walk trees (e.g. a menu or org chart).

```sql
-- Sales report: seats sold and revenue per event (tested on our database)
WITH sold AS (                                    -- step 1: totals from bookings
    SELECT EVT_ID, SUM(QTY) AS SOLD_QTY, SUM(TOT_AMT) AS REVENUE
    FROM TB_BOOKING
    WHERE STS_CD = 'BOOKED'
    GROUP BY EVT_ID
)
SELECT e.EVT_NM, e.TOT_SEAT,                       -- step 2: join with events
       COALESCE(s.SOLD_QTY, 0) AS SOLD,
       COALESCE(s.REVENUE, 0)  AS REVENUE
FROM TB_EVENT e LEFT JOIN sold s ON s.EVT_ID = e.EVT_ID
ORDER BY REVENUE DESC;
```

```
        evt_nm         | tot_seat | sold | revenue      (real output)
 Phnom Penh Tech Summit |        5 |    4 |  160.00
 Rule 1 · only 5 seats  |        5 |    4 |  100.00
```

In our project: not needed: our queries are simple CRUD. A CTE is useful for reports like this, run as a native query (`@Query(value = "WITH ...", nativeQuery = true)`).

Remember: "CTE = `WITH` + a named sub-result, to split a complex query into readable steps."

Q Transaction and ACID

A transaction is a group of SQL that succeeds or fails **together**.

- **A**tomic: all or nothing
- **C**onsistent: rules/constraints stay true
- **I**solated: parallel transactions don't mix
- **D**urable: after commit, it's saved

Ours: `@Transactional` on `book()` : reduce seats + insert booking, or neither.

Q Why NUMERIC for money, VARCHAR(14) for dates?

NUMERIC(12,2) is exact. FLOAT/DOUBLE has rounding errors (0.1 + 0.2 ≠ 0.3), which is unacceptable for money. In Java we use `BigDecimal` for the same reason.

VARCHAR(14) `YYYYMMDDHH24MISS` is the company convention from slide 22. It sorts correctly as text. `DateTimes` converts it to `LocalDateTime` .

C API

Process flow, parameters, methods, status codes

Q What is the process flow of an API request?

Client JSON + token → **Tomcat** port 8080 → **Security filter** check JWT → 401/403 → **DispatcherServlet** find controller → **Controller** @Valid → 400 → **Service** @Transactional, rules → 409 → **Repository** JPA → SQL → **PostgreSQL**

On the way back: entity → **Mapper** → DTO → Jackson → JSON response with a status code. Any exception goes to `GlobalExceptionHandler`, which returns the same JSON error format (ProblemDetail). Full step-by-step: section 5.

> **Remember:** "Security → Controller → Service → Repository → DB, and back as a DTO. Each layer can stop the request with its own error code."

Q Path variable vs request param (query param) vs request body

	@PathVariable	@RequestParam	@RequestBody
Where	Inside the URL path	After `?` in the URL	JSON in the request body
Looks like	`/api/events/12`	`/api/events?status=ACTIVE&page=0`	`{"eventId":12,"quantity":2}`
Use for	**Which** resource (ID)	**Filter, sort, page** (optional)	**Data** to create / update
Required?	Yes, part of the URL	Often optional (`required = false`)	Yes for POST / PUT
Ours	`GET /api/bookings/{id}`	`GET /api/events?status=ACTIVE`	`POST /api/bookings`

```
@GetMapping("/{id}")                                    // EventController
public EventResponse get(@PathVariable Long id) {...}

@GetMapping
public Page<EventResponse> list(@RequestParam(required = false) EventStatus status, Pageable
pageable) {...}

@PostMapping                                           // BookingController
public BookingResponse book(@Valid @RequestBody CreateBookingRequest request) {...}
```

"Path param" and "path variable" are the same idea: Spring calls it `@PathVariable`, JAX-RS (Jakarta REST) calls it `@PathParam`.

> **Remember:** "Path = which one, query = how to filter, body = the data."

Q HTTP methods

Method	Does	Idempotent?
GET	Read	Yes
POST	Create / action	No
PUT	Replace whole	Yes
PATCH	Change part	Not always
DELETE	Remove	Yes

Idempotent = sending it twice has the same effect as once. **Why `POST /bookings/{id}/cancel` not DELETE?** We keep the row with status CANCELLED for history; it's an action, not a removal.

Q Status codes we return

200 OK	Read / update / cancel worked
201 Created	New event or booking
400 Bad Request	Invalid input (quantity 5)
401 Unauthorized	Not logged in / bad token
403 Forbidden	Logged in, not allowed
404 Not Found	Event 999
409 Conflict	Valid, but state says no (no seats)
500 Server Error	Bug; logged, no stack trace sent

D Querying data

How many ways, and which we used

Q How many ways can you query data from the database?

Slide 11 teaches the **3 main ways** in Spring Data JPA. There are more tools beyond them:

#	Way	Example	Use when	In our project
1	**Derived query** method name = query	`findByUsername(String username)` `findByStatus(status, pageable)`	Simple filters	used UserRepository, EventRepository, BookingRepository
2	**JPQL** `@Query` uses entity names	`@Query("select e from Event e where e.id = :id")`	Joins, custom conditions, locks	used `findByIdForUpdate` (+ `@Lock`)
3	**Native SQL** real table names	`@Query(value = "WITH ...", nativeQuery = true)`	DB features, CTE, reports	not needed
4	**Criteria API / Specification**	`spec.where(hasStatus(..)).and(priceBelow(..))`	Search screens with many optional filters	not needed
5	**Projection** (DTO / interface)	`select new OrderView(o.id, c.name) ...`	Read only some columns	not needed
6	**JdbcTemplate / JdbcClient**	`jdbc.query("select ...", rowMapper)`	Plain SQL, no entities	not needed
7	**QueryDSL / jOOQ / MyBatis**	Type-safe or XML-mapped SQL libraries	Big projects, complex SQL	not needed

Plus `@EntityGraph` (used in `BookingRepository`): not a separate way, but it tells JPA to load related data in the same query.

> **Remember:** "Three main ways: derived method, JPQL, native SQL. I used derived queries for simple lookups and JPQL with a lock for booking. Criteria, projections and JdbcTemplate exist for special cases."

Q What is the N + 1 problem?

Load 20 bookings with 1 query, then each `booking.getEvent()` runs 1 more query → **21 queries**.

Ours: `@EntityGraph(attributePaths = {"event", "user"})` loads bookings with their event and user in **1 query** (JOIN). Plus `default_batch_fetch_size: 50` as a safety net.

Q LAZY vs EAGER

LAZY: load the related object only when you use it.
EAGER: always load it immediately.

Ours: `@ManyToOne(fetch = LAZY)` everywhere (EAGER causes hidden extra queries), then `@EntityGraph` exactly where we need the data.

Remember: "LAZY by default, fetch what you need with @EntityGraph or JOIN FETCH."

E Two requests at the same time

The race condition behind rule 1

Q What goes wrong without protection?

Event has **1 seat**. Alice and Bob both book 1 ticket at the same moment:

Time	Alice's request	Bob's request
t1	reads AVL_SEAT = 1 ✔	
t2		reads AVL_SEAT = 1 ✔
t3	1 ≥ 1 → saves AVL_SEAT = 0	
t4		1 ≥ 1 → saves AVL_SEAT = 0
result	**2 tickets sold for 1 seat: oversold ("lost update")**	

5 ways to handle it

#	Way	How	Good / bad
1	**Pessimistic lock** ✅ ours	`@Lock(PESSIMISTIC_WRITE)` → `SELECT ... FOR UPDATE`. Bob's request **waits** until Alice's transaction commits, then sees 0 seats → 409.	Simple and always correct. Requests on the same event wait in line (fine for ticketing).
2	**Optimistic lock**	Add a `@Version` column. Update fails if someone changed the row since you read it → retry or 409.	No waiting; good when conflicts are rare. Popular events have many conflicts → many retries.
3	**Atomic UPDATE**	`UPDATE TB_EVENT SET AVL_SEAT = AVL_SEAT - :qty WHERE EVT_ID = :id AND AVL_SEAT >= :qty` → 0 rows updated = no seats → 409.	Fastest, one SQL. Rule logic moves into SQL instead of the entity. (Tested: returns `UPDATE 0` when not enough seats.)
4	**DB constraint** ✅ ours too	`CHECK (AVL_SEAT >= 0)`: the database refuses negative seats.	Last safety net; gives an ugly error alone, so combine with 1–3.
5	**Isolation SERIALIZABLE / distributed lock**	Strictest DB isolation level, or a Redis lock when many servers/databases are involved.	Strong but heavier; needed for multi-server systems with shared resources.

```java
// EventRepository: our solution
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select e from Event e where e.id = :id")
Optional<Event> findByIdForUpdate(@Param("id") Long id);
```

Proof: `ConcurrentBookingTest` sends **20 bookings at the same time** for a 5-seat event → exactly **5 succeed, 15 get 409**, seats end at 0.

Remember: "I lock the event row with SELECT FOR UPDATE, so the second request waits and then sees the real seat count. The CHECK constraint is a backup. Optimistic locking or an atomic UPDATE would also work."

F Security config

Who are you, and what may you do

Q Authentication vs authorization

Authentication = who are you? (login, check token) → fails with **401**.

Authorization = what may you do? (role, ownership) → fails with **403**.

Remember: "401 = I don't know you. 403 = I know you, but no."

Q What's inside our JWT?

`header . payload . signature` (3 parts, Base64):

```
{ "sub": "alice", "uid": 2,
  "roles": ["USER"],
  "iat": 1790000000, "exp": 1790003600 }
```

Signed with **HS256** + secret from `JWT_SECRET`. Anyone can *read* it, nobody can *change* it without the secret. So never put passwords in a JWT.

Q How is security configured? (SecurityConfig)

```
http.csrf(csrf -> csrf.disable())                          // no cookies → no CSRF
  .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))  // no session
  .authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**", "/error").permitAll()     // login
    .requestMatchers("/", "/index.html", "/swagger-ui/**").permitAll()
    .requestMatchers(GET, "/api/events/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers("/api/events/**").hasRole("ADMIN")        // write
    .requestMatchers("/api/bookings/**").hasAnyRole("USER", "ADMIN")
    .anyRequest().authenticated())
  .oauth2ResourceServer(o -> o.jwt(...))       // check Bearer token
```

Piece	Why
BCrypt passwords	One-way hash with salt, deliberately slow, so stolen hashes are hard to crack
Same error for wrong user / wrong password	Attackers can't find out which usernames exist
Secret in environment variable	Never in code or Git; app refuses to start if it's shorter than 32 bytes
Stateless + JWT	No session stored on the server; any server can check the token → easy to scale
CSRF disabled	CSRF attacks abuse cookies; we don't use cookies
Ownership check in service	Roles can't express "only your booking", so `BookingServiceImpl` checks owner → 403
Errors as ProblemDetail	401/403 from the filter are forwarded to `GlobalExceptionHandler` for the same JSON format

> **Remember:** "Stateless JWT: login gives a signed token with the roles, every request is checked by the filter chain, URL rules check the role, and the service checks ownership."

G Code structure and patterns

Where things live and why

Q Code structure

```
com.kosign.eventtickets
├── config/          security, swagger, JWT
props
├── controller/      HTTP only
├── service/         interfaces (what)
│   └── impl/        implementations (how)
├── repository/      database access
├── domain/entity/   tables + business rules
│   └── enums/
├── dto/request/     what API receives
├── dto/response/    what API returns
├── mapper/          entity ⇄ DTO
├── exception/       errors → JSON
├── security/        current user from JWT
└── util/            date helper
```

Q Why this structure?

- Each folder has **one job**, so it's easy to find code.
- Controllers stay thin; rules live in entities and services.
- Controller depends on a service **interface** → easy to test or replace.
- DTOs hide the database shape; mappers keep conversion in one place.
- Matches the layered pattern from slide 9 and common team style.

> **Remember:** "Layer by type: controller → service → impl → repository, with DTO, mapper and entity."

Design patterns used

Pattern	Where in our code	Why
Layered architecture	controller → service → repository	Separation of concerns
Dependency Injection / IoC	Constructor injection with `@RequiredArgsConstructor`	Spring creates and wires objects; loose coupling
Repository	`EventRepository extends JpaRepository`	Hide SQL behind an interface
DTO	`CreateBookingRequest`, `BookingResponse` (records)	Control what goes in and out
Mapper	`EventMapper`, `BookingMapper`	One place for entity ⇄ DTO
Program to an interface	`BookingService` / `BookingServiceImpl`	Swap or mock the implementation
Rich domain model	`Event.reserveSeats()`, `Booking.cancel()`	Rules can't be skipped
Builder	`JwtClaimsSet.builder()...build()`	Build an object step by step
Chain of responsibility	Spring Security filter chain	Each filter checks then passes on
Proxy (AOP)	`@Transactional` opens/commits around the method	Cross-cutting code without repeating it
Singleton	Every Spring bean (one instance)	Shared, stateless services
Centralized error handling	`@RestControllerAdvice GlobalExceptionHandler`	Same error JSON everywhere

Rapid-fire answers

What does @Transactional do?
Runs the method in one DB transaction: commit on success, rollback on exception.

What is Flyway?
Versioned SQL migrations run at startup, once each; history in flyway_schema_history.

What is a Bean? IoC?
An object Spring creates and manages. IoC = Spring controls object creation, not my code.

ddl-auto: validate?
Hibernate only checks entities match tables; Flyway creates them.

PUT vs PATCH?
PUT replaces the whole resource; PATCH changes some fields. We use PUT for event update.

open-in-view: false?
DB access only inside services; no hidden lazy queries while writing JSON.

Why DTO, not entity, in the API?
Hides password and audit fields, stable API, client can't send fields like price.

Why 409, not 400, for no seats?
The request is valid; it conflicts with the current state of the event.

What does @Valid do?
Runs the annotations (@Min, @NotBlank…) on the request; failure → 400 with field errors.

Why 403, not 404, for another's booking?
The homework asks for 403. Some teams use 404 to hide that the ID exists.

Why store UNIT_PRC in booking?
If the price changes later, old bookings keep the price actually paid.

Token expired?
After 1 hour the filter returns 401; the user logs in again (refresh tokens would be next).

What is ProblemDetail?
Standard JSON error format (RFC 9457): title, status, detail, instance.

JAR vs WAR?
JAR has Tomcat inside; WAR can also go into an external Tomcat. Ours is WAR (requirement) and still runs with java -jar.

How do you test?
MockMvc tests on H2 (21), live run on PostgreSQL, the demo page, Swagger.

Why Lombok?
Generates getters and constructors, less boilerplate code.

What is CORS?
Browser rule for calls to another domain. Not needed here: the demo page is served by the same app.

What would you add next?
Refresh tokens, auto-cancel bookings when an event is cancelled, Testcontainers, payment.

Golden rule: answer in one sentence first, then point to the code in the project. If you don't know, say "I didn't use that here, but I understand it as…" rather than guessing.